Riemannian World Models: Deep Reinforcement Learning on Data-Driven Differentiable Manifolds

Etale Cohomology

March 2026

Abstract

The central contribution of this paper is the identification and formal separation of two distinct Riemannian manifolds that coexist in every Deep Reinforcement Learning system, and the demonstration that prior geometric DRL research has addressed only the first while systematically ignoring the second.

The **first manifold** $\mathcal{M}_{\text{weights}}$ is the policy parameter space, equipped with the Fisher information metric $G_F(\theta)_{ij}$. This manifold governs *how* an agent learns. The Natural Policy Gradient (NPG; Kakade, 2001) and its descendants—TRPO and PPO—correctly handle the geometry of this manifold, and have done so for over two decades. The **second manifold** $\mathcal{M}_{\text{env}}$ is the environmental state space, equipped with the pullback metric $g_{ij}(z) = (J_{f_\theta}^\top J_{f_\theta})_{ij}$ derived from the Geometric Intelligence (GI) Theory VAE pipeline. This manifold governs *where* an agent acts. Every existing World Models architecture—DreamerV3, DreamerV2, DreamerV1, PlaNet, and all related model-based RL methods—treats $\mathcal{M}_{\text{env}}$ as Euclidean, an assumption that is geometrically incorrect whenever the environmental state space is intrinsically curved.

These two manifolds are structurally distinct in every dimension: the first manifold $\mathcal{M}_{\text{weights}}^{(i)}$ is a statistical manifold (one copy per agent, with Fisher metric), while the second manifold $\mathcal{M}_{\text{env}}$ is a data-driven Riemannian manifold constructed from industry-specific business data (one shared copy for all agents, with pullback metric). Conflating them—or addressing only the first while ignoring the second—constitutes a fundamental geometric error in the foundations of model-based DRL applied to real-world socioeconomic environments.

We present **Riemannian World Models (RWM)**, the first framework to correctly handle the geometry of $\mathcal{M}_{\text{env}}$ in a World Models DRL setting. We identify four structural errors that Euclidean World Models commit on curved $\mathcal{M}_{\text{env}}$: (1) the transition model performs Euclidean vector addition that moves the agent off the manifold; (2) the policy gradient uses the Euclidean gradient rather than the Riemannian gradient $g^{ij}\partial_j V$; (3) temporal credit assignment compares gradient vectors across different tangent spaces without Levi-Civita parallel transport; (4) the reward function uses Euclidean distance rather than the intrinsic geodesic distance d_g. We prove geometrically correct replacements for each error, establish their consistency (Theorem 4.1), and provide PyTorch-compatible implementations via automatic differentiation from the VAE decoder. We further explain the four-layer structural reason why neither the social science, engineering, nor machine learning communities have previously identified this gap, and provide a roadmap for extension to Finsler manifolds—required for aviation routing, wind energy control, and maritime logistics—where the metric is directionally asymmetric.

Contents

1 Introduction **3**
1.1 The Problem: Agents Walking Off the Manifold 3
1.2 Why This Matters: The GI Theory Pipeline 3
1.3 Why Business and Policy Environments Are Curved 4
1.4 Contributions 4

2 Background **4**
2.1 The GI Theory Manifold Construction 4
2.2 Why $\mathcal{M}_{\text{env}}$ Is Riemannian, Not Pseudo-Riemannian 5
2.3 World Models and DreamerV3 5
2.4 The Two-Manifold Structure of DRL: The Central Novelty of This Work . 5

3 The Four Geometric Errors of Euclidean World Models **7**
3.1 Error 1: Euclidean State Transition 7
3.2 Error 2: Euclidean Policy Gradient 7
3.3 Error 3: Trivial Parallel Transport 8
3.4 Error 4: Euclidean Distance Reward 8

4 Riemannian World Models **8**
4.1 Correction 1: Exponential Map Transition Model 8
4.2 Correction 2: Riemannian Policy Gradient 9
4.3 Correction 3: Levi-Civita Parallel Transport 9
4.4 Correction 4: Geodesic Distance Reward 10
4.5 Integrated Architecture 10
4.6 Geometric Consistency Theorem 11

5 Why This Framework Has Not Appeared in Prior Literature **12**
5.1 Layer 1: Social Science — Data Sparsity and Educational Gap 12
5.2 Layer 2: Engineering and Physics — The Known-Manifold Assumption . . 12
5.3 Layer 3: Machine Learning — The Two-Manifold Confusion 12
5.4 Layer 4: The Disciplinary Intersection Problem 13

6 Multi-Agent Extension **13**

7 Future Work: Finsler Manifolds and Time-Varying Finsler Manifolds **14**
7.1 Finsler Manifolds 14
7.2 Time-Varying Finsler Manifolds 14
7.3 Industrial Problem Settings Requiring Finsler Geometry 14
7.4 Mathematical Open Problems 15
7.5 Research Roadmap 15

8 Related Work **16**

9 Conclusion **16**

A Notation Summary **18**

B Geometric Framework Comparison **18**

1 Introduction

1.1 The Problem: Agents Walking Off the Manifold

Modern Deep Reinforcement Learning has achieved remarkable success across diverse domains, from game playing to robotic control. The dominant paradigm for model-based DRL—World Models [Ha & Schmidhuber, 2018, Hafner et al., 2023]—learns a latent representation of the environment and plans within that representation. However, every major World Models architecture rests on an unexamined geometric assumption: **the latent state space is Euclidean**.

Concretely, the transition model in DreamerV3 [Hafner et al., 2023] computes

$$z_{t+1} = z_t + f_\theta(z_t, a_t) + \varepsilon \tag{1}$$

where the addition $z_t + f_\theta(\cdot)$ is the ordinary vector addition of $\mathbb{R}^n$. When the true state space is a curved manifold $\mathcal{M}_{\text{env}}$, this operation is geometrically ill-defined: the sum $z_t + f_\theta(z_t, a_t)$ may lie *outside* $\mathcal{M}_{\text{env}}$ entirely—analogous to adding a displacement vector to a point on the surface of the Earth and obtaining a point floating above it.

This error propagates into: the policy gradient computation (which should use the Riemannian gradient $g^{ij}\partial_j V$ rather than the Euclidean gradient $\partial_j V$); temporal credit assignment (which requires Levi-Civita parallel transport to compare gradients across different tangent spaces); and the reward function (which should use geodesic distance rather than Euclidean distance).

1.2 Why This Matters: The GI Theory Pipeline

The Geometric Intelligence (GI) theory [Etale Cohomology, 2024a,b] provides a concrete pipeline for constructing the state space $\mathcal{M}_{\text{env}}$ of a corporate management environment or national policy environment as a **proper Riemannian manifold**. The pipeline proceeds as follows:

1. Collect industry-specific business data $\{x_1, \ldots, x_N\} \subset \mathbb{R}^D$ (financial indicators, market prices, macroeconomic variables, operational metrics, etc.)
2. Train a VAE with a C^∞ decoder $f_\theta : \mathbb{R}^d \to \mathbb{R}^D$ (using tanh activations to guarantee smoothness)
3. Verify that the Jacobian $J_{f_\theta}(z)$ has full rank d for all z in the region of interest (Proposition 2.1)
4. Compute the pullback metric $g_{ij}(z) = (J_{f_\theta}^\top J_{f_\theta})_{ij}$, guaranteed positive definite by the full-rank condition
5. Derive Christoffel symbols, curvature tensors, Lie derivatives, and geodesics from g_{ij}

The resulting space $(\mathcal{M}_{\text{env}}, g)$ is a genuine Riemannian manifold sharing the same mathematical structure as the spacetime manifolds of general relativity—with the same toolkit (covariant derivatives, geodesic equations, curvature tensors) but a positive definite rather than Lorentzian metric. **The GI pipeline guarantees Riemannian structure regardless of the industry, data distribution, or statistical properties of the input**; it is the construction procedure, not the data content, that determines the manifold type.

1.3 Why Business and Policy Environments Are Curved

A natural question arises: why should a corporate management or policy environment be modeled as a curved manifold rather than Euclidean space? We offer three complementary answers.

Structural nonlinearity of interactions. The relationship between macroeconomic indicators, market prices, and corporate performance is fundamentally nonlinear. When a company operates near a phase transition—transitioning from growth to contraction, from a supply-constrained to a demand-constrained regime, or from one regulatory environment to another—small changes in state variables produce qualitatively different responses. In Riemannian terms, these phase transitions correspond to regions of **high scalar curvature** $\mathrm{Scal}(z) = g^{ij}\,\mathrm{Ric}_{ij}$: neighborhoods where the metric tensor changes rapidly, geodesics diverge, and Euclidean approximations break down most severely.

Anisotropy of state sensitivity. The pullback metric $g_{ij}(z)$ at each point $z \in \mathcal{M}_{\text{env}}$ measures how sensitive the high-dimensional data space is to changes in the latent state in each direction. In directions where the business environment is sensitive (e.g., near a credit rating threshold, or at peak demand season), g_{ij} is large; in insensitive directions, g_{ij} is small. This spatial anisotropy is precisely the information encoded in the Christoffel symbols—information that Euclidean DRL discards.

Data manifold hypothesis. The high-dimensional data generated by a corporate environment does not fill $\mathbb{R}^D$ uniformly; it concentrates near a low-dimensional submanifold [Fefferman et al., 2016]. The VAE decoder learns a faithful parametrization of this manifold, and the pullback metric quantifies its intrinsic geometry. Proposition 2.1 ensures this geometry is Riemannian under three natural conditions.

1.4 Contributions

1. **Problem formalization.** We formally characterize the four geometric errors made by Euclidean World Models when applied to Riemannian state spaces constructed via the GI Theory pipeline.
2. **Riemannian corrections.** We propose four geometrically correct replacements and prove their consistency (Theorem 4.3).
3. **Implementation.** We provide PyTorch-compatible implementations of all four corrections, computable via automatic differentiation from the VAE decoder.
4. **Gap analysis.** We identify and explain the four-layer structural reason why this framework has not appeared in prior literature.
5. **Future directions.** We provide a structured roadmap for extension to Finsler and time-varying Finsler manifolds, with concrete industrial motivation.

2 Background

2.1 The GI Theory Manifold Construction

Proposition 2.1 (GI Theory, Vol. 2)**.** *Let $f_\theta : U \subset \mathbb{R}^d \to \mathbb{R}^D$ be the decoder of a VAE trained on business data, satisfying:*

(i) $f_\theta \in C^\infty(U)$ (achieved by using tanh, GELU, or Softplus activations; ReLU is excluded)

(ii) $\operatorname{rank} J_{f_\theta}(z) = d$ *for all* $z \in U$ (verified numerically via SVD)

(iii) f_θ *is a proper map* (compact preimages of compact sets are compact)

Then $\mathcal{M}_{env} := f_\theta(U)$ *is a* d*-dimensional embedded submanifold of* $\mathbb{R}^D$*, and the pullback metric*

$$g_{ij}(z) = \sum_{k=1}^{D} \frac{\partial f_\theta^k}{\partial z^i}(z)\, \frac{\partial f_\theta^k}{\partial z^j}(z) = (J_{f_\theta}^\top J_{f_\theta})_{ij} \tag{2}$$

is a Riemannian metric on U*.*

Proof sketch. Condition (i) ensures f_θ is a smooth immersion. Condition (ii) ensures nondegeneracy: for all $v \neq 0$,

$$v^\top g(z) v = v^\top J_{f_\theta}(z)^\top J_{f_\theta}(z) v = \|J_{f_\theta}(z) v\|^2 > 0$$

since $\operatorname{rank}(J_{f_\theta}) = d$ implies $J_{f_\theta} v \neq 0$ for $v \neq 0$. Thus g_{ij} is positive definite. Condition (iii) upgrades the immersion to a proper embedding via the standard theorem. □

Remark 2.2 (Industry-independence)**.** *Positive definiteness of* g_{ij} *follows entirely from the algebraic structure* $J^\top J$*, independent of the industry or data distribution. This is the mathematical fact that guarantees Riemannian structure across all GI Theory applications—manufacturing, finance, energy, national security—uniformly.*

2.2 Why $\mathcal{M}_{\text{env}}$ Is Riemannian, Not Pseudo-Riemannian

General relativity uses a pseudo-Riemannian (Lorentzian) metric with signature $(-,+,+,+)$, allowing the metric to take negative values in the time direction. The GI Theory manifold $\mathcal{M}_{\text{env}}$ carries a **positive definite** metric: $g(v,v) > 0$ for all nonzero tangent vectors v. The reason is structural: business and policy data have no analog to the time-space asymmetry of physics. Every direction of change in the business environment has positive "distance." The $J^\top J$ construction algebraically enforces this.

Despite the difference in metric signature, both frameworks share the same differential-geometric toolkit: Levi-Civita connection, covariant derivatives, Christoffel symbols, Riemann curvature tensor, and geodesic equations. The formulas are identical; only the sign convention of the metric differs.

2.3 World Models and DreamerV3

DreamerV3 [Hafner et al., 2023] implements the World Models paradigm via a Recurrent State Space Model (RSSM):

$$z_t \sim q_\phi(z_t \mid h_t, o_t), \quad h_{t+1} = f_\psi(h_t, z_t, a_t), \quad z_{t+1} \sim p_\phi(z_{t+1} \mid h_{t+1})$$

All operations are performed in $\mathbb{R}^n$. Actor and critic networks are trained via truncated backpropagation through imagined trajectories. This is appropriate for flat state spaces (Atari, DMControl, Minecraft) but geometrically incorrect when $\mathcal{M}_{\text{env}}$ is curved.

2.4 The Two-Manifold Structure of DRL: The Central Novelty of This Work

The core observation of this paper is that every DRL system simultaneously involves two distinct Riemannian manifolds, and that prior work has addressed only the first. We state this precisely.

The First Manifold: Policy Parameter Space $\mathcal{M}^{(i)}_{\textbf{weights}}$

Natural Policy Gradient (NPG; Kakade 2001) recognizes that the policy parameter space carries a natural Riemannian structure. For a parameterized policy π_θ, the Fisher information matrix

$$G_F(\theta)_{ij} = \mathbb{E}_{s,a\sim\pi_\theta}\left[\frac{\partial \log \pi_\theta(a|s)}{\partial \theta^i}\frac{\partial \log \pi_\theta(a|s)}{\partial \theta^j}\right] \tag{3}$$

defines a Riemannian metric on the space of policy parameters. This statistical manifold $(\mathcal{M}_{\text{weights}}, G_F)$ governs *how the agent learns.* The NPG update

$$\theta \leftarrow \theta + \alpha\, G_F(\theta)^{-1}\nabla_\theta J(\theta) \tag{4}$$

computes the Riemannian gradient on $\mathcal{M}_{\text{weights}}$—the direction of steepest ascent in the Fisher metric, not the Euclidean gradient. This is the correct geometric operation on the first manifold.

The lineage of algorithms that correctly handle the first manifold:

Natural Gradient (Amari, 1998) $\rightarrow$ NPG (Kakade, 2001) $\rightarrow$ TRPO (Schulman et al., 2015) $\rightarrow$ PPO (S

DreamerV3 uses PPO for its actor update and therefore *does* correctly handle the geometry of the first manifold via the Fisher metric.

Key property of the first manifold. The Riemannian gradient of a scalar function $J(\theta)$ on $(\mathcal{M}_{\text{weights}}, G_F)$ requires only the metric inverse G_F^{-1}—no Christoffel symbols are needed:

$$(\text{grad}_{G_F} J)^i = G_F^{ij}(\theta)\,\partial_j J(\theta)$$

This is because the covariant derivative of a scalar field does not involve the connection. Christoffel symbols appear only in the covariant derivative of vector fields, which arises only on the *second* manifold.

The Second Manifold: Environmental State Space $\mathcal{M}_{\textbf{env}}$

The GI Theory pipeline constructs a second, fundamentally different Riemannian manifold: the environmental state space $\mathcal{M}_{\text{env}}$ equipped with the pullback metric $g_{ij}(z) = (J_{f_\theta}^\top J_{f_\theta})_{ij}$. This manifold governs *where the agent acts and moves.* Unlike the first manifold—whose metric G_F is derived from the policy's statistical properties—the second manifold's metric is derived from the geometry of the data itself, via the Jacobian of the VAE decoder.

Every existing World Models architecture treats $\mathcal{M}_{\textbf{env}}$ as Euclidean. DreamerV3's transition model (1) performs Euclidean vector addition $z_t + f_\theta(z_t, a_t)$, implicitly assuming $\Gamma^k_{ij} = 0$ at every point of $\mathcal{M}_{\text{env}}$. This assumption holds only when $\mathcal{M}_{\text{env}}$ is flat—a condition that generically fails for manifolds constructed from real business or policy data.

The second manifold requires Christoffel symbols explicitly, because the agent's movement on $\mathcal{M}_{\text{env}}$ involves:

- **Transitions between states**: vectors in different tangent spaces $T_{z_t}\mathcal{M}_{\text{env}}$ and $T_{z_{t+1}}\mathcal{M}_{\text{env}}$ cannot be compared without the connection
- **Parallel transport**: gradient information must be transported along trajectories on $\mathcal{M}_{\text{env}}$ using Γ^k_{ij}
- **Exponential map**: the transition model must map tangent vectors back to points on $\mathcal{M}_{\text{env}}$ via the geodesic equation $\ddot{z}^k + \Gamma^k_{ij}\dot{z}^i\dot{z}^j = 0$

The Fundamental Distinction

Table 1 summarizes the two-manifold structure. In a multi-agent system with N agents, there are $N+1$ manifolds in total: one $\mathcal{M}_{\text{env}}$ shared by all agents, and N individual copies of $\mathcal{M}^{(i)}_{\text{weights}}$, one per agent.

Table 1: The two Riemannian manifolds in DRL: only the first has been addressed by prior work

	First Manifold $\mathcal{M}^{(i)}_{\text{weights}}$	**Second Manifold $\mathcal{M}_{\text{env}}$**
Elements	Policy parameters $\theta_i \in \mathbb{R}^p$	Environment states $z \in \mathbb{R}^d$
Metric	Fisher information $G_F(\theta)$	Pullback metric $(J_{f_\theta}^\top J_{f_\theta})_{ij}$
Metric source	Statistical properties of π_θ	Jacobian of VAE decoder f_θ
Role	"How agent i learns"	"Where all agents act"
Multiplicity	N copies (one per agent)	1 shared by all N agents
Needs Christoffel?	No (scalar gradient only)	Yes (transitions, transport, geodesics)
Prior treatment	NPG $\to$ TRPO $\to$ PPO ✓	**This work** (all prior work: ignored)
DreamerV3 handling	Correct (PPO actor update)	**Incorrect** (assumes Euclidean)

Remark 2.3 (Why NPG is insufficient)**.** *NPG and its descendants (TRPO, PPO) correctly handle the first manifold but say nothing about the second. An agent using PPO on $(\mathcal{M}_{weights}, G_F)$ while operating on a curved $\mathcal{M}_{env}$ correctly updates its policy parameters in the Fisher-metric sense—but then executes those policy updates using a geometrically incorrect transition model, Euclidean gradient, and Euclidean distance reward. The geometric correctness of the first manifold does not compensate for the geometric incorrectness of the second. Both must be handled simultaneously.*

Remark 2.4 (The novelty of this work)**.** *To our knowledge, no prior work has (a) identified the two-manifold structure as a fundamental feature of DRL, (b) shown that all existing World Models architectures correctly handle the first manifold while systematically ignoring the second, or (c) proposed geometrically correct modifications to World Models to address the second manifold. This is the primary contribution of this paper.*

3 The Four Geometric Errors of Euclidean World Models

3.1 Error 1: Euclidean State Transition

$$z_{t+1} = z_t + f_\theta(z_t, a_t) \quad \text{(Euclidean addition — geometrically incorrect on curved } \mathcal{M}_{\text{env}}\text{)} \tag{5}$$

The tangent vector $f_\theta(z_t, a_t) \in T_{z_t}\mathcal{M}_{\text{env}}$ added to z_t via Euclidean addition generically produces a point outside $\mathcal{M}_{\text{env}}$. The per-step error magnitude is $O(\kappa\|f_\theta\|^3)$ where κ is the sectional curvature at z_t.

3.2 Error 2: Euclidean Policy Gradient

$$\nabla V = \partial_j V \quad \text{(Euclidean gradient — geometrically incorrect on curved } \mathcal{M}_{\text{env}}\text{)} \tag{6}$$

The correct steepest-ascent direction on $(\mathcal{M}_{\text{env}}, g)$ is the Riemannian gradient $g^{ij}(z)\partial_j V(z)$, which coincides with the Euclidean gradient only when $g_{ij} = \delta_{ij}$.

3.3 Error 3: Trivial Parallel Transport

Gradient vectors live in different tangent spaces: $\nabla V(z_{t+1}) \in T_{z_{t+1}}\mathcal{M}_{\text{env}}$ and $\nabla V(z_t) \in T_{z_t}\mathcal{M}_{\text{env}}$. Euclidean DRL identifies them via trivial transport of $\mathbb{R}^n$, which is geometrically incorrect on curved manifolds. Comparing vectors in different tangent spaces without parallel transport is analogous to comparing velocity vectors of a particle on a sphere using ambient Cartesian coordinates.

3.4 Error 4: Euclidean Distance Reward

$$r = -\|z_{t+1} - z^*\|^2 \quad \text{(Euclidean distance — not the intrinsic metric of } \mathcal{M}_{\text{env}}) \tag{7}$$

The geodesic distance $d_g(z_{t+1}, z^*)$ is the intrinsically correct distance measure on $(\mathcal{M}_{\text{env}}, g)$, measuring the length of the shortest curve lying within $\mathcal{M}_{\text{env}}$ that connects the two points.

4 Riemannian World Models

4.1 Correction 1: Exponential Map Transition Model

Definition 4.1 (Exponential map)**.** *The exponential map at $z_t \in \mathcal{M}_{env}$ is $\exp_{z_t} : T_{z_t}\mathcal{M}_{env} \to \mathcal{M}_{env}$ defined by*

$$\exp_{z_t}(v) = \gamma(1)$$

where γ is the unique geodesic satisfying $\gamma(0) = z_t$, $\dot\gamma(0) = v$, solving:

$$\ddot\gamma^k + \Gamma^k_{ij}(\gamma)\,\dot\gamma^i\dot\gamma^j = 0 \tag{8}$$

with Christoffel symbols

$$\Gamma^k_{ij}(z) = \frac{1}{2}\,g^{kl}(z)(\partial_i g_{jl}(z) + \partial_j g_{il}(z) - \partial_l g_{ij}(z)) \tag{9}$$

Corrected transition model:

$$\boxed{z_{t+1} = \exp_{z_t}(f_\theta(z_t, a_t))} \tag{10}$$

Properties. (a) $\exp_{z_t}(v) \in \mathcal{M}_{\text{env}}$ by definition (Hopf-Rinow theorem for complete manifolds). (b) When $\Gamma^k_{ij} = 0$ (flat space): $\exp_{z_t}(v) = z_t + v$, recovering (1). (c) The name derives from S^1: $\exp_{e^{i\theta}}(v) = e^{i\theta} \cdot e^{iv}$.

Since $\mathcal{M}_{\text{env}}$ is constructed via a VAE decoder without a closed-form exponential map, we solve the geodesic ODE (8) numerically via 4th-order Runge-Kutta. The Christoffel symbols (9) are computed via automatic differentiation of $g_{ij}(z) = J_{f_\theta}(z)^\top J_{f_\theta}(z)$.

Listing 1: Christoffel symbols and exponential map via automatic differentiation

```
def christoffel_symbols(z, decoder):
    """Compute Gamma^k_{ij}(z) via autodiff of g_{ij} = J^T J."""
    d = z.shape[0]
    z = z.detach().requires_grad_(True)
    J = torch.autograd.functional.jacobian(decoder, z)  # (D, d)
    g = J.T @ J          # pullback metric (d, d)
```

```
    g_inv = torch.linalg.inv(g + 1e-6 * torch.eye(d))
    dg = torch.zeros(d, d, d)   # dg[k,i,j] = d_k g_{ij}
    for i in range(d):
        for j in range(d):
            grad = torch.autograd.grad(
                g[i, j], z, create_graph=True)[0]
            dg[:, i, j] = grad
    # Gamma^k_{ij} = (1/2) g^{kl}(d_i g_{jl}+d_j g_{il}-d_l g_{ij})
    Gamma = torch.zeros(d, d, d)
    for k in range(d):
        for i in range(d):
            for j in range(d):
                for l in range(d):
                    Gamma[k,i,j] += 0.5 * g_inv[k,l] * (
                        dg[i,j,l] + dg[j,i,l] - dg[l,i,j])
    return Gamma, g, g_inv

def exponential_map(z, v, Gamma, steps=20):
    """exp_z(v) via 4th-order Runge-Kutta on geodesic ODE."""
    def rhs(state):
        pos, vel = state[:len(z)], state[len(z):]
        d = pos.shape[0]
        acc = torch.zeros(d)
        for k in range(d):
            for i in range(d):
                for j in range(d):
                    acc[k] -= Gamma[k,i,j] * vel[i] * vel[j]
        return torch.cat([vel, acc])
    dt = 1.0 / steps
    state = torch.cat([z, v])
    for _ in range(steps):
        k1 = rhs(state);   k2 = rhs(state + 0.5*dt*k1)
        k3 = rhs(state + 0.5*dt*k2);  k4 = rhs(state + dt*k3)
        state = state + (dt/6.0)*(k1 + 2*k2 + 2*k3 + k4)
    return state[:len(z)]
```

4.2 Correction 2: Riemannian Policy Gradient

$$\boxed{\mathrm{grad}_g V(z) = g^{ij}(z)\,\partial_j V(z)} \tag{11}$$

Remark 4.2. *The Riemannian gradient of a scalar field does not require Christoffel symbols—only the metric inverse g^{ij}. Christoffel symbols appear only in covariant derivatives of vector fields (Corrections 1 and 3).*

Listing 2: Riemannian gradient of scalar value function

```
def riemannian_gradient(V_func, z, g_inv):
    """grad_g V = g^{ij} d_j V."""
    z = z.requires_grad_(True)
    V = V_func(z)
    eucl_grad = torch.autograd.grad(V, z, create_graph=True)[0]
    return g_inv @ eucl_grad
```

4.3 Correction 3: Levi-Civita Parallel Transport

To propagate gradient information from z_{t+1} back to z_t during temporal credit assignment, we solve the Levi-Civita parallel transport equation along the trajectory:

$$\boxed{\frac{DV^k}{dt} = \frac{dV^k}{dt} + \Gamma^k_{ij}(\gamma)\,\dot{\gamma}^i V^j = 0} \tag{12}$$

Listing 3: Levi-Civita parallel transport of gradient vector

```
def parallel_transport(V, z_start, z_end, Gamma_func, steps=20):
    """Transport V from z_start to z_end along the geodesic."""
    dt = 1.0 / steps
    z_cur, V_cur = z_start.clone(), V.clone()
    z_dot = (z_end - z_start)   # geodesic velocity (linear approx.)
    for _ in range(steps):
        Gamma = Gamma_func(z_cur)
        d = V_cur.shape[0]
        V_dot = torch.zeros(d)
        for k in range(d):
            for i in range(d):
                for j in range(d):
                    V_dot[k] -= Gamma[k,i,j] * z_dot[i] * V_cur[j]
        z_cur = z_cur + dt * z_dot
        V_cur = V_cur + dt * V_dot
    return V_cur
```

4.4 Correction 4: Geodesic Distance Reward

$$d_g(z, z^*) = \inf_{\gamma} \int_0^1 \sqrt{g_{ij}(\gamma(t))\, \dot{\gamma}^i(t)\, \dot{\gamma}^j(t)}\, dt \tag{13}$$

$$\boxed{r(z_t, a_t) = -d_g(z_{t+1}, z^*)^2} \tag{14}$$

The Bellman equation $V(z_t) = r(z_t, a_t) + \gamma V(z_{t+1})$ retains its standard form; only the reward uses the intrinsic geodesic distance (13).

4.5 Integrated Architecture

Listing 4: RiemannianWorldModelAgent: integrated architecture

```
class RiemannianWorldModelAgent(nn.Module):
    """
    World Models DRL agent on Riemannian manifold M_env.
    Requires GI Theory VAE decoder satisfying Proposition 2.1.
    """
    def __init__(self, decoder, latent_dim, action_dim):
        super().__init__()
        self.decoder = decoder   # f_theta: R^d -> R^D, C^inf, full-
            rank
        self.d = latent_dim
        self.transition_net = nn.Sequential(  # outputs v in T_{z_t}
            M_env
            nn.Linear(latent_dim + action_dim, 256), nn.Tanh(),
            nn.Linear(256, 256), nn.Tanh(),
            nn.Linear(256, latent_dim))
        self.value_net = nn.Sequential(
            nn.Linear(latent_dim, 256), nn.Tanh(), nn.Linear(256, 1))
        self.policy_net = nn.Sequential(
            nn.Linear(latent_dim, 256), nn.Tanh(),
            nn.Linear(256, action_dim))

    def step(self, z_t, a_t):
        """Correction 1: z_{t+1} = exp_{z_t}(f_theta(z_t, a_t))."""
        Gamma, g, g_inv = christoffel_symbols(z_t, self.decoder)
        v = self.transition_net(torch.cat([z_t, a_t]))
        return exponential_map(z_t, v, Gamma)
```

```python
    def policy_loss(self, z_t):
        """Correction 2: policy gradient via Riemannian gradient of V.
            """
        _, g, g_inv = christoffel_symbols(z_t, self.decoder)
        z_t = z_t.requires_grad_(True)
        V_t = self.value_net(z_t)
        eucl_grad = torch.autograd.grad(V_t, z_t, create_graph=True)
            [0]
        riem_grad = g_inv @ eucl_grad
        return -torch.dot(self.policy_net(z_t), riem_grad)

    def bellman_loss(self, z_t, a_t, z_next, z_goal, gamma=0.99):
        """Correction 4: reward via geodesic distance."""
        def metric(z): return christoffel_symbols(z, self.decoder)[1]
        r = -geodesic_distance(z_next, z_goal, metric)
        V_t = self.value_net(z_t)
        V_next = self.value_net(z_next).detach()
        return (V_t - (r + gamma * V_next)) ** 2

    def train_step(self, trajectory, z_goal):
        """Correction 3: temporal credit with Levi-Civita transport.
            """
        total_loss, V_grad = 0.0, None
        for t in reversed(range(len(trajectory) - 1)):
            z_t, a_t = trajectory[t]
            z_next, _ = trajectory[t+1]
            if V_grad is not None:
                V_grad = parallel_transport(
                    V_grad, z_next, z_t,
                    lambda z: christoffel_symbols(z, self.decoder)[0])
            total_loss += self.bellman_loss(z_t, a_t, z_next, z_goal)
        return total_loss
```

4.6 Geometric Consistency Theorem

Theorem 4.3 (Geometric consistency of RWM)**.** *Under Proposition 2.1, the Riemannian World Model agent satisfies:*

(a) ***Manifold preservation.*** $\exp_{z_t}(v) \in \mathcal{M}_{env}$ *for all* $v \in T_{z_t}\mathcal{M}_{env}$ *with* $\|v\|_g$ *sufficiently small (Hopf-Rinow theorem).*

(b) ***Coordinate independence.*** *The Riemannian gradient* $g^{ij}\partial_j V$*, the covariant derivative* DV^k/dt*, and the geodesic distance* d_g *are tensorial expressions invariant under changes of coordinate chart.*

(c) ***Flat-space recovery.*** *When* $g_{ij} = \delta_{ij}$ *(flat metric,* $\Gamma^k_{ij} = 0$*): Correction 1 recovers* $z_{t+1} = z_t + f_\theta$*; Correction 2 recovers* $\partial_j V$*; Correction 3 reduces to trivial transport; Correction 4 recovers Euclidean distance. DreamerV3 behavior is exactly recovered.*

(d) ***Steepest ascent.*** *The Riemannian gradient* $\mathrm{grad}_g V$ *is the unique direction of steepest ascent of* V *on* $(\mathcal{M}_{env}, g)$ *of unit* g*-norm.*

Proof. (a) follows from the definition of the exponential map and Hopf-Rinow. (b) follows from the tensorial transformation laws of g^{ij}, $\partial_j V$, and Γ^k_{ij} under coordinate changes. (c) is immediate from $\Gamma^k_{ij} = 0$ when $g = \delta$. (d) follows from the characterization of the Riemannian gradient as the metric dual of the differential dV: $\langle \mathrm{grad}_g V, w\rangle_g = dV(w)$ for all $w \in T_z\mathcal{M}_{\mathrm{env}}$. □

5 Why This Framework Has Not Appeared in Prior Literature

The absence of this framework from prior work reflects four structural barriers, not accidental oversight.

5.1 Layer 1: Social Science — Data Sparsity and Educational Gap

The GI Theory problem setting was computationally infeasible before the VAE era. Corporate financial data is collected at quarterly or annual frequency, producing sparse discrete time series that cannot support the smooth, dense sampling required for differentiable manifold construction. The VAE [Kingma & Welling, 2013] provided, for the first time, a principled method for learning a C^∞ parametrization of a data manifold from finite samples.

Beyond computational feasibility, graduate curricula in economics, business administration, and policy science do not include differential geometry or Riemannian manifolds as core requirements. The conceptual vocabulary of "curvature," "geodesic," and "covariant derivative" is absent from the training of most social science researchers, preventing the formulation of curved-space models of management and policy environments.

5.2 Layer 2: Engineering and Physics — The Known-Manifold Assumption

Researchers in robotics and control theory who work with Riemannian manifolds typically operate on **manifolds with known, fixed geometric structure**: $SO(3)$ for rigid body rotation, $SE(3)$ for pose estimation, S^2 for direction vectors, symmetric positive definite matrices for covariance.

The GI Theory manifold $\mathcal{M}_{\text{env}}$ is fundamentally different: its geometry is **unknown a priori and learned from data**. No analytical expression for the Christoffel symbols is available; everything must be computed numerically via automatic differentiation of $g_{ij}(z) = J_{f_\theta}(z)^\top J_{f_\theta}(z)$.

This data-driven, unknown-geometry setting falls outside the standard problem formulation of geometric control. Roboticists ask: "Given that the configuration space is $SO(3)$, how do we control?" The GI Theory question is: "Given industry data from an unknown domain, what is the geometry of the state space, and how do we learn an optimal policy on it?" These are categorically different problems.

5.3 Layer 3: Machine Learning — The Two-Manifold Confusion

Machine learning researchers who have introduced Riemannian geometry into reinforcement learning have almost universally addressed the **first manifold** (policy parameter space):

- NPG [Kakade, 2001]: Fisher metric on policy parameters $\mathcal{M}_{\text{weights}}$
- TRPO [Schulman et al., 2015]: trust region constraint in Fisher geometry of $\mathcal{M}_{\text{weights}}$
- PPO [Schulman et al., 2017]: clipped approximation of the NPG constraint on $\mathcal{M}_{\text{weights}}$
- Müller & Montúfar [2023]: gradient flow on the occupancy measure manifold (related to $\mathcal{M}_{\text{weights}}$)

This body of work is mathematically correct and important. However, it addresses only one of the two manifolds. The second manifold $\mathcal{M}_{\text{env}}$—where the agent actually moves and acts—has received almost no geometric treatment in the model-based RL literature.

The reason is structural: the machine learning training loop is organized around parameter updates, making the geometry of $\mathcal{M}_{\text{weights}}$ naturally salient. The state space $\mathcal{M}_{\text{env}}$ is treated as a fixed Euclidean background. This is the *two-manifold confusion*: researchers correctly applied Riemannian geometry to the first manifold and then concluded that the geometric problem was solved—without recognizing that a second, qualitatively different manifold was being left unaddressed.

The confusion is compounded by the benchmark problem. DreamerV3's standard benchmarks (Atari, DMControl, Minecraft) have state spaces that are well-approximated by Euclidean geometry, so the geometric error on $\mathcal{M}_{\text{env}}$ never becomes visible in practice. The curvature effects that matter—phase transitions in business cycles, regime changes in policy environments, threshold nonlinearities in operational data, near a credit rating boundary or a market liquidity threshold—are precisely the features of GI Theory application domains that are absent from standard RL benchmarks. The error was invisible in the environments where the algorithms were tested, which prevented its identification.

5.4 Layer 4: The Disciplinary Intersection Problem

The full framework simultaneously requires:

1. Differential geometry and Riemannian manifolds (mathematics)
2. Variational autoencoders and smooth manifold learning (machine learning)
3. Deep reinforcement learning and World Models (RL engineering)
4. Industry-specific business and policy data structures (applied domain)

As of March 2026, the community of researchers fluent in all four simultaneously is extremely small. The problem has fallen through disciplinary cracks—visible only from a vantage point simultaneously inside all four fields, which the GI Theory research program has developed through its case study methodology across manufacturing, finance, shipping, aviation, and national security domains.

6 Multi-Agent Extension

The GI Theory deployment context involves N heterogeneous agents simultaneously on $\mathcal{M}_{\text{env}}$. The RWM framework extends directly:

$$\mathcal{M}_{\text{env}} : \text{one shared Riemannian manifold for all agents}$$

$$\mathcal{M}^{(i)}_{\text{weights}} : \text{individual Fisher-metric manifold for agent } i, \quad i = 1, \ldots, N$$

Inter-agent communication with parallel transport. When agents at different positions $z_i, z_j \in \mathcal{M}_{\text{env}}$ exchange gradient information (message vectors $m_{ij}(t)$), the message from agent j must be parallel-transported along the geodesic from z_j to z_i before combining with agent i's local information:

$$\frac{d\theta_i}{dt} = -\nabla_{\theta_i}\mathcal{L}_i + \sum_{j\in\mathcal{N}(i)} \alpha_{ij} \cdot \tau_{z_j\to z_i}(m_{ij}(t)) \tag{15}$$

where $\tau_{z_j\to z_i}$ denotes Levi-Civita parallel transport (12) from z_j to z_i. Without this transport, combining gradients from different tangent spaces of $\mathcal{M}_{\text{env}}$ is geometrically incorrect.

7 Future Work: Finsler Manifolds and Time-Varying Finsler Manifolds

7.1 Finsler Manifolds

A **Finsler manifold** (M, F) generalizes Riemannian geometry by replacing the metric with a norm function $F : TM \to [0, \infty)$ satisfying:

$$F(x, \lambda v) = \lambda F(x, v) \text{ for } \lambda > 0, \qquad F(x, v) > 0 \text{ for } v \neq 0$$

The fundamental tensor $g_{ij}(x, v) = \frac{1}{2}\frac{\partial^2 F^2}{\partial v^i \partial v^j}$ is positive definite for all $v \neq 0$ but depends on *both position and velocity.* Crucially, $F(x, v) \neq F(x, -v)$ in general—the metric is *directionally asymmetric.* The Riemannian metric is the special case $F(x, v) = \sqrt{g_{ij}(x)v^i v^j}$ with no velocity dependence.

The Finsler Christoffel symbols $G^k_{ij}(x, v)$ are velocity-dependent, making computations substantially more expensive than the Riemannian case.

7.2 Time-Varying Finsler Manifolds

When the Finsler metric itself varies with time, we obtain a **time-varying Finsler manifold**:

$$F(x, v, t): \text{ the norm depends on position, velocity, } \textit{and} \text{ time}$$

The geodesic equation acquires an additional forcing term:

$$\ddot{\gamma}^k + G^k_{ij}(\gamma, \dot{\gamma}, t)\, \dot{\gamma}^i \dot{\gamma}^j = -\frac{\partial G^k}{\partial t}(\gamma, \dot{\gamma}, t) \tag{16}$$

No DRL or World Models framework has addressed this structure as of March 2026.

7.3 Industrial Problem Settings Requiring Finsler Geometry

7.3.1 Aviation: Optimal Routing Under Wind

The effective ground speed of an aircraft flying with heading v at position x in a wind field $w(x)$ is:

$$F_{\text{air}}(x, v) = \|v\|_{\text{air}} + \langle w(x), \hat{v} \rangle, \qquad \hat{v} = v/\|v\|$$

A tailwind reduces flight time; a headwind increases it. This creates $F_{\text{air}}(x, v) \neq F_{\text{air}}(x, -v)$—a fundamental Finsler asymmetry that cannot be captured by any Riemannian metric. Optimal routing (minimizing fuel or time) is a geodesic computation on a Finsler manifold.

Time-varying structure. Jet stream winds vary seasonally (stronger and more northerly in winter at mid-latitudes). A fully correct agent requires $F_{\text{air}}(x, v, t)$ where t encodes the seasonal phase.

7.3.2 Wind Energy: Turbine Control Under Directional Wind

The wake effect in a wind farm creates directional asymmetry: wind flowing from an upstream turbine degrades the downstream turbine's performance, but not vice versa. The "distance" between operating states of a turbine farm is inherently asymmetric in the wind direction, requiring a Finsler metric.

Time-varying structure. North Sea wind speed and direction exhibit strong seasonal variation (substantially stronger winter winds) and diurnal variation (sea-breeze cycles). Optimal pitch control and yaw alignment must operate on $F_{\text{wind}}(x, v, t)$ where t encodes seasonal and diurnal phase.

7.3.3 Maritime Routing

Ocean currents (Gulf Stream, Kuroshio, Antarctic Circumpolar Current) create strong directional asymmetries in ship travel times. The Gulf Stream significantly reduces westbound Atlantic crossing time compared to eastbound. The state space of maritime navigation is Finsler, not Riemannian.

7.3.4 Supply Chain and Logistics

Transportation network costs are asymmetric (delivery vs. empty return) and time-dependent (traffic patterns, seasonal demand peaks). The state space of a logistics optimization agent exhibits Finsler structure with strong time-variation.

7.4 Mathematical Open Problems

P1. Finsler transition model. Define and implement $\exp^F_{z_t}(v)$ for a data-driven Finsler manifold constructed from aviation or wind data, where the exponential map depends on v even in its linear approximation.

P2. Finsler gradient. Define the Finsler gradient of a scalar function V via the Legendre transform of F, and implement it via automatic differentiation.

P3. Finsler parallel transport. Define parallel transport using the Chern or Rund connection (both velocity-dependent), and implement for DRL temporal credit assignment.

P4. Convergence on time-varying Finsler. Prove convergence of a World Models DRL agent on $F(x, v, t)$, where the environment is non-stationary at the geometric level.

P5. Data-driven Finsler construction. Develop a pipeline analogous to the GI Theory VAE pipeline for constructing a Finsler manifold from data with directional asymmetry.

7.5 Research Roadmap

Table 2: Research roadmap for Riemannian and Finsler World Models

Horizon	Framework	Mathematical challenge	Industrial application
1–2 years	Riemannian World Models (this work)	Autodiff of $J^\top J$; geodesic ODE	Management, policy environments
2–4 years	Finsler World Models	Velocity-dependent $G^k_{ij}(x, v)$; Legendre transform gradient	Aviation routing, maritime
4+ years	Time-varying Finsler	Time-derivative forcing in geodesic eq.; non-stationary learning	Wind turbine control, logistics
Theoretical	Convergence analysis	Gradient flow on data-driven manifolds	All domains

8 Related Work

Riemannian optimization. Absil et al. [2008] and Boumal [2023] develop optimization on matrix manifolds ($SO(n)$, Stiefel, Grassmann) with known, fixed structure. Bonnabel [2013] establishes SGD convergence on Riemannian manifolds. All address parameter-space geometry (first manifold).

Natural policy gradient and the first manifold. Kakade [2001] introduces NPG, applying the Fisher information metric to the policy parameter space—the first manifold $\mathcal{M}_{\text{weights}}$. Schulman et al. [2015, 2017] develop TRPO and PPO as practical approximations. Müller & Montúfar [2023] proves convergence via Hessian geometry of the occupancy measure manifold. This entire lineage correctly handles $\mathcal{M}_{\text{weights}}$ and only $\mathcal{M}_{\text{weights}}$. Crucially, DreamerV3 [Hafner et al., 2023] inherits first-manifold correctness from its PPO actor update—but simultaneously treats $\mathcal{M}_{\text{env}}$ as Euclidean in its world model transition. This paper identifies and corrects the latter.

DRL on structured manifolds. Ratliff et al. [2018] design Riemannian Motion Policies on $SO(3)$ and $SE(3)$. Liu et al. [2022] formulate hierarchical RL subgoal search as Riemannian manifold optimization. Both use manifolds with known, fixed geometry. Neither identifies the two-manifold structure, and both implicitly treat the environmental state space as the only manifold in the problem.

Latent space geometry. Arvanitidis et al. [2018] demonstrate non-Euclidean geometry in VAE latent spaces via pullback metrics—the closest prior work to our treatment of $\mathcal{M}_{\text{env}}$. They address generative modeling traversal, not DRL policy learning, and do not identify the two-manifold structure. We extend their pullback metric insight to the DRL regime.

Finsler geometry. Bao et al. [2000] provide the standard mathematical reference. No prior work applies Finsler geometry to DRL; the two-manifold structure identified here extends naturally to the Finsler case as future work.

GI Theory. Etale Cohomology [2024a,b] establish the VAE manifold construction pipeline and optimal control framework via the Pontryagin maximum principle on $\mathcal{M}_{\text{env}}$. The present work introduces DRL and World Models into GI Theory for the first time, and explicitly identifies the two-manifold structure as the foundational novelty distinguishing this work from all prior geometric DRL research.

9 Conclusion

We have presented Riemannian World Models, the first framework to correctly handle the geometry of the *second manifold* $\mathcal{M}_{\text{env}}$—the environmental state space—in a World Models DRL setting.

The primary contribution is the identification, formalization, and resolution of a structural gap that has persisted in the geometric DRL literature since Natural Policy Gradient (Kakade, 2001): **every DRL system simultaneously involves two distinct Riemannian manifolds, and all prior geometric DRL research has addressed only the first**. The first manifold $\mathcal{M}_{\text{weights}}$ (policy parameter space with Fisher metric) governs how the agent learns; NPG, TRPO, and PPO correctly handle its geometry, and DreamerV3 inherits this correctness for its actor update. The second manifold $\mathcal{M}_{\text{env}}$ (environmental state space with pullback metric $J^\top J$) governs where the agent acts; every existing World Models architecture treats it as Euclidean, committing four structural geometric errors in the transition model, policy gradient, temporal credit assignment,

and reward function.

These two manifolds are not interchangeable. They differ in their metric source (statistical vs. data-driven), their multiplicity (N individual copies vs. one shared copy), their need for Christoffel symbols (absent vs. required), and their prior treatment in the literature (twenty years of active research vs. none). Addressing the first while ignoring the second is not a partial solution—it is a solution to a different problem.

The four-layer gap analysis explains why this structure went unrecognized: the data and educational constraints of social science prevented the construction of curved socioeconomic state spaces; the known-manifold assumption of engineering prevented the recognition of data-driven unknown-geometry manifolds as a valid problem setting; the two-manifold confusion of machine learning led researchers to believe that NPG/TRPO/PPO had resolved the geometric problem; and the disciplinary intersection required to see all layers simultaneously was achievable only from the vantage point of the GI Theory research program.

The extension to Finsler manifolds—required for aviation routing, wind energy control, maritime logistics, and supply chain optimization under directional asymmetry—and to time-varying Finsler manifolds remains a significant open problem. But the two-manifold structure identified here applies to the Finsler case as well: the first manifold remains the policy parameter space; the second manifold becomes a Finsler manifold. The problem is now precisely formulated. The boundary between the geometrically tractable and the geometrically open is more clearly drawn than before.

References

Absil, P.-A., Mahony, R., & Sepulchre, R. (2008). *Optimization Algorithms on Matrix Manifolds*. Princeton University Press.

Amari, S.-I. (1998). Natural gradient works efficiently in learning. *Neural Computation*, 10(2), 251–276.

Amari, S.-I. (2016). *Information Geometry and Its Applications*. Springer.

Arvanitidis, G., Hansen, L. K., & Hauberg, S. (2018). Latent space oddity: On the curvature of deep generative models. *International Conference on Learning Representations (ICLR)*.

Bao, D., Chern, S.-S., & Shen, Z. (2000). *An Introduction to Riemann-Finsler Geometry*. Springer.

Bonnabel, S. (2013). Stochastic gradient descent on Riemannian manifolds. *IEEE Transactions on Automatic Control*, 58(9), 2217–2229.

Boumal, N. (2023). *An Introduction to Optimization on Smooth Manifolds*. Cambridge University Press.

Bronstein, M. M., Bruna, J., LeCun, Y., Szlam, A., & Vandergheynst, P. (2017). Geometric deep learning: Going beyond Euclidean data. *IEEE Signal Processing Magazine*, 34(4), 18–42.

do Carmo, M. P. (1992). *Riemannian Geometry*. Birkhäuser.

Etale Cohomology. (2024a). *Geometric Intelligence, Volume 1: AI Agents, Optimal Control, and Lie Derivatives on Management Environment Manifolds*. Zenodo. https://doi.org/10.5281/zenodo.19140918

Etale Cohomology. (2024b). *Geometric Intelligence, Volume 2: Mathematical Foundations of Differentiable Manifolds from Business Data*. Zenodo. https://doi.org/10.5281/zenodo.19157891

Fefferman, C., Mitter, S., & Narayanan, H. (2016). Testing the manifold hypothesis. *Journal of the American Mathematical Society*, 29(4), 983–1049.

Ha, D., & Schmidhuber, J. (2018). World models. *arXiv preprint* arXiv:1803.10122.

Hafner, D., Pasukonis, J., Ba, J., & Lillicrap, T. (2023). Mastering diverse domains with world models. *arXiv preprint* arXiv:2301.04104.

Kakade, S. M. (2001). A natural policy gradient. *Advances in Neural Information Processing Systems*, 14.

Kingma, D. P., & Welling, M. (2013). Auto-encoding variational Bayes. *International Conference on Learning Representations (ICLR 2014)*.

Liu, S., Qiao, P., Dou, Y., & Jin, R. (2022). Searching latent sub-goals in hierarchical reinforcement learning as Riemannian manifold optimization. *IEEE International Conference on Multimedia and Expo (ICME)*.

Meta AI. (2021). Deep Riemannian manifold learning. *arXiv preprint* arXiv:2110.07458.

Milosevic, N., Müller, J., & Montúfar, G. (2025). The geometry of nonlinear reinforcement learning. *Workshop on Geometry and Topology in Machine Learning, NeurIPS 2025*.

Müller, J., & Montúfar, G. (2023). Geometry and convergence of natural policy gradient methods. *Information Geometry*, 6, 485–526.

Ratliff, N. et al. (2018). Riemannian motion policies. *arXiv preprint* arXiv:1801.02854.

Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015). Trust region policy optimization. *International Conference on Machine Learning (ICML)*.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint* arXiv:1707.06347.

A Notation Summary

B Geometric Framework Comparison

Table 3: Mathematical notation used throughout the paper

Symbol	**Definition**
$\mathcal{M}_{\text{env}}$	Environmental state space (second manifold, Riemannian)
$\mathcal{M}_{\text{weights}}^{(i)}$	Policy parameter manifold for agent i (first manifold, Fisher metric)
$f_\theta : \mathbb{R}^d \to \mathbb{R}^D$	VAE decoder (C^∞, full-rank Jacobian)
$g_{ij}(z) = (J^\top J)_{ij}$	Pullback Riemannian metric on $\mathcal{M}_{\text{env}}$
$g^{ij}(z)$	Inverse metric tensor
$\Gamma^k_{ij}(z)$	Levi-Civita Christoffel symbols
$\exp_{z_t}(v)$	Riemannian exponential map at z_t applied to v
$d_g(z, z^*)$	Geodesic distance between z and z^*
$\text{grad}_g V = g^{ij}\partial_j V$	Riemannian gradient of scalar function V
DV^k/dt	Covariant derivative of vector field V along a curve
$G_F(\theta)$	Fisher information matrix (first manifold metric)
$F(x, v)$	Finsler metric function (velocity-dependent)
$F(x, v, t)$	Time-varying Finsler metric
$G^k_{ij}(x, v)$	Finsler Christoffel symbols (velocity-dependent)

Table 4: Comparison of geometric frameworks addressed in this paper

Framework	**Metric**	**Velocity dep.**	**Time dep.**	**DRL status**
Euclidean $\mathbb{R}^n$	δ_{ij} (flat)	No	No	Standard DreamerV3
Riemannian (M, g)	$g_{ij}(x) > 0$	No	No	**This work**
Pseudo-Riemannian	$g_{ij}(x)$ indef.	No	No	Not applicable (GR)
Finsler (M, F)	$g_{ij}(x, v) > 0$	Yes	No	Open problem
Time-var. Finsler	$g_{ij}(x, v, t) > 0$	Yes	Yes	Open problem